java code quality

- Java is one of the most popular programming languages, powering everything from enterprise systems to web applications and beyond.
- Writing high-quality Java code is important for keeping applications reliable, scalable, and efficient.
- One way to improve code quality is by using static analysis tools.
- These tools scan the code without running it, helping developers catch bugs, security risks, and structural issues early in the development process

Fewer Bugs and Errors

- Java applications often power critical systems in industries like banking, healthcare, and e-commerce.
- A single bug in these environments can lead to financial losses, security breaches, or a loss of trust.
- Writing clear and structured code while following best practices reduces the risk of errors and helps maintain system stability.
-

Measuring Code Quality

- **Cyclomatic Complexity:** Measures how complicated a method is by counting the number of independent paths through the code. Simpler code is usually easier to maintain and test.
- **Code Coverage:** Shows how much of the codebase is tested by automated tests. A higher percentage means fewer untested areas that could hide potential issues.
- **Technical Debt:** Represents the extra work needed to fix shortcuts taken during development. Keeping technical debt low helps prevent future headaches.
- **Code Duplication:** Repeating code makes maintenance harder and increases the chance of bugs. Tools that detect duplication help developers stick to the DRY (Don’t Repeat Yourself) principle.
-

Static Code Analysis Tools

- **Checkstyle:** Enforces coding standards and formatting rules.
- **PMD:** Detects common mistakes and potential bugs.
- **SonarQube:** Provides a dashboard to track code quality over time and manage technical debt.
- **FindBugs/SpotBugs:** Analyzes code patterns to spot potential bugs before they cause problems.
-

Checkstyle

- Checkstyle helps developers write Java code that follows a consistent format and structure.
- It automates style enforcement, making it useful for teams that need to follow a specific coding standard.
- **Features**
  - **Code Style Enforcement:** Helps maintain a uniform coding style across a project by checking for spacing, naming conventions, and formatting rules.
  - **Custom Rules:** Developers can define custom rules to match specific project needs.
  - **Integration with Build Tools:** Works with Maven, Gradle, and IDEs like Eclipse and IntelliJ IDEA to provide real-time feedback.
-

PMD

- PMD scans Java source code to detect potential issues such as unused variables, inefficient object creation, and other performance-related concerns.
- **Features**
  - **Bug Detection:** Finds common mistakes like empty catch blocks, redundant object creation, and unused variables.
  - **Predefined and Custom Rules:** Comes with built-in rule sets and allows developers to create their own rules.
  - **CI/CD Integration:** Works with continuous integration tools and popular IDEs to run automatic code checks.
-

Maven PMD Plugin to your pom.xml:

```
<project>
  <build>
  <plugins>
   <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-pmd-plugin</artifactId>
    <version>3.22.0</version> <!-- Use a recent version -->
    <configuration>
     <rulesets>
      <ruleset>/rulesets/java/maven-pmd-plugin-default.xml</ruleset>
      <!-- You can add custom rulesets or other PMD rulesets here -->
      <!-- <ruleset>/path/to/your/custom-ruleset.xml</ruleset> -->
     </rulesets>
     <targetJdk>1.8</targetJdk> <!-- Specify your Java version -->
     <printFailingErrors>true</printFailingErrors>
     <failOnViolation>true</failOnViolation> <!-- Optional: Fail the build on PMD violations -->
    </configuration>
<execution>
      <goals>
       <goal>pmd</goal>
       <goal>cpd</goal> <!-- Optional: Run Copy/Paste Detector (CPD) -->
      </goals>
     </execution>
    </executions>
   </plugin>
  </plugins>
 </build>
 ...
</project>
```

Running pmd

Running PMD:
Once configured, you can run PMD as part of your Maven build.
To run PMD and generate a report:
Code

```
mvn pmd:pmd
```

This will generate a PMD report, typically in target/site/pmd.html.

To run PMD as part of your standard build lifecycle (e.g., install):

```
mvn clean install
```

FindBugs/SpotBugs

- Originally known as FindBugs, SpotBugs is an open-source tool that analyzes Java code to find patterns that may indicate errors or performance problems.
- **Features**
  - **Pattern-Based Analysis:** Detects a variety of issues, from logic errors to performance bottlenecks.
  - **Detailed Explanations:** Provides descriptions of detected issues to help developers understand and resolve them.
  - **Extendable with Plugins:** Supports additional plugins to cover specific use cases.
-

```
package com.example;
public class BuggyClass {
    private String name;
    public BuggyClass(String name) {
        this.name = name;
    }
    public boolean equals(BuggyClass other) { // Potential bug: should override Object.equals(Object)
        if (other == null) {
            return false;
        }
        return this.name.equals(other.name);
    }

    public static void main(String[] args) {
        BuggyClass obj1 = new BuggyClass("Test");
        BuggyClass obj2 = new BuggyClass("Test");
        System.out.println(obj1.equals(obj2)); // Will print true
        System.out.println(obj1.equals(new Object())); // Will cause a ClassCastException if the method was truly overridden
    }
}
```

SpotBugs in Maven

```
<project>
<build>
    <plugins>
      <plugin>
        <groupId>com.github.spotbugs</groupId>
        <artifactId>spotbugs-maven-plugin</artifactId>
        <version>4.8.3</version> <!-- Use the latest stable version -->
        <configuration>
          <effort>Max</effort>
          <threshold>Low</threshold>
          <failOnError>true</failOnError>
          <xmlOutput>true</xmlOutput>

<xmlOutputDirectory>${project.build.directory}/spotbugs</xmlOutput
Directory>
        </configuration>
<executions>
          <execution>
            <goals>
              <goal>check</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

mvn clean install

Explanation of Configuration:

- effort: Sets the analysis effort level (Min, Less, Default, More, Max). Higher effort finds more bugs but takes longer.
- threshold: Sets the minimum bug priority to report (Low, Medium, High).
- failOnError: If true, the build will fail if bugs above the specified threshold are found.
- xmlOutput: Generates an XML report of the findings.
- xmlOutputDirectory: Specifies the directory for the XML report.
- check goal: Executes the SpotBugs analysis during the Maven build lifecycle.

Run SpotBugs:

Execute the Maven build.

Code

mvn clean install

After the build, you can find the generated XML report in target/spotbugs/spotbugsXml.xml (or your configured directory). This report details the identified bug patterns, their severity, a

SonarQube

- SonarQube is a comprehensive code analysis tool that continuously monitors code quality. It supports multiple programming languages, including Java, and provides insights into bugs, vulnerabilities, and code structure.
- **Features**
  - **Quality Checks:** Allows developers to set conditions that must be met before code is approved.
  - **Technical Debt Tracking:** Estimates how much time is needed to fix detected issues.
  - **Code Metrics:** Measures complexity, duplication, reliability, and security concerns.
  - **User Dashboard:** Offers a visual overview of code health over time.
-

Maven with SonarQube

integrate SonarQube with a Spring Boot project using Maven, you would add the SonarQube Maven plugin to your pom.xml:

Code

```
<build>
  <plugins>
    <plugin>
      <groupId>org.sonarsource.scanner.maven</groupId>
      <artifactId>sonar-maven-plugin</artifactId>
      <version>3.9.1.2184</version> <!-- Use a recent version -->
    </plugin>
  </plugins>
</build>
```

Running SpotBugs with Maven

- setting up SpotBugs with Maven
- ```
  <build>
    <plugins>
      <plugin>
        <groupId>com.github.spotbugs</groupId>
        <artifactId>spotbugs-maven-plugin</artifactId>
        <version>4.0.0</version>
        <executions>
          <execution>
            <goals>
              <goal>check</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
  ```
- SpotBugs runs automatically during the Maven build, helping to catch potential issues early in the development cycle
  ```

JaCoCo

- An open-source tool for measuring Java code coverage, which reports on how much of your code is executed by your tests.
- It works by instrumenting Java code at runtime through a Java agent, collecting data during test execution, and then generating reports that show metrics like line, branch, and method coverage.
- JaCoCo can be integrated with build tools like Maven and Gradle to automate the process and is supported by various CI/CD tools for continuous monitoring.

How it works

- Instrumentation: JaCoCo uses a Java agent to instrument Java class files on-the-fly as they are loaded by the Java Virtual Machine (JVM).
- Data collection: While the tests are running, the agent tracks which lines of code, methods, and branches are executed.
- Report generation: After the tests are finished, JaCoCo gathers the collected execution data and generates reports in formats like HTML, XML, or CSV. These reports detail the percentage of code that was covered by tests.

Key features

- Build tool integration: It offers plugins for build tools such as Maven and Gradle, which automate the configuration and execution process.
- Versatile agent: The JaCoCo agent can be used in different modes, such as writing execution data to a local file, or outputting it over a TCP socket for other tools to retrieve.
- Wide adoption: It's a de facto standard for code coverage in Java and Android projects.
- Continuous integration: It integrates with CI/CD platforms like GitLab and Jenkins, allowing for automated reports and quality gates in the development pipeline.

JaCoCo maven plugin in pom.xml file.

```
<build>
  <pluginManagement>
    <plugins>
      <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <version>0.8.8</version>
      </plugin>
    </plugins>
  </pluginManagement>
  <plugins>
    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <executions>
        <execution>
          <goals>
            <goal>prepare-agent</goal>
          </goals>
        </execution>
        <execution>
          <id>report</id>
          <phase>test</phase>
          <goals>
            <goal>report</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

Example

```
public class ShippingService {
    public int calculateShippingFee(int weight) {
        if (weight <= 0) {
            throw new
IllegalStateException("Please provide correct
weight");
        }
        if (weight <= 2) {
            return 5;
        } else if (weight <= 5) {
            return 10;
        }
        return 15;
    }
}
```

- present a method for calculating shipping fees. The input parameter is the weight, which is used to determine the shipping fee

Test class

- public class TestShippingService {
  @Test
  public void incorrectWeight() {
  ShippingService shippingService = new ShippingService();
  assertThrows(IllegalStateException.class, () ->
  shippingService.calculateShippingFee(-1));
  }

  @Test
  public void firstRangeWeight() {
  ShippingService shippingService = new ShippingService();
  assertEquals(5, shippingService.calculateShippingFee(1));
  }
  }
-

Testing

- Go to Maven, select clean and test commands then Run Maven Build
- Once the test has been completed, target/site/jacoco/index.html contains all output. Let's open that file in Browser to see the details.

jacoco

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
io.truongbn.github.jacoco		60%		50%	3	7	5	11	1	4	0	2
Total	12 of 30	60%	3 of 6	50%	3	7	5	11	1	4	0	2

Jacoco with sonar

- **JaCoCo Plugin:** Add the jacoco-maven-plugin to generate code coverage reports.

```
<build>
    <plugins>
      <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <version>0.8.11</version> <!-- Use a recent version --
>
        <executions>
          <execution>
            <id>prepare-agent</id>
            <goals>
              <goal>prepare-agent</goal>
            </goals>
</execution>
            <execution>
              <id>report</id>
              <goals>
                <goal>report</goal>
              </goals>
            </execution>
          </executions>
        </plugin>
        <!-- ... other plugins -->
    </plugins>
</build>
```

SonarQube Plugin

- Include the sonar-maven-plugin to facilitate integration with SonarQube.

```
<build>
    <plugins>
      <!-- ... jacoco plugin ... -->
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.10.0.2594</version> <!-- Use a recent version -->
      </plugin>
    </plugins>
  </build>
```

- **SonarQube Properties**
- Configure SonarQube connection details and specify the JaCoCo report path.

```
<properties>
    <sonar.projectKey>your_project_key</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
<!-- Adjust if SonarQube is elsewhere -->
    <sonar.login>your_sonar_token</sonar.login> <!--
Generate a token in SonarQube -->

<sonar.coverage.jacoco.xmlReportPaths>${project.build.directory}/jacoco/jacoco.xml</sonar.coverage.jacoco.xmlReportPaths>
  </properties>
```

-

Generating Reports and Analysis:

- Build and Run Tests: Execute your project's tests to generate JaCoCo coverage data.
- mvn clean install

- SonarQube Analysis: Run the SonarQube analysis to send the JaCoCo report and other code quality metrics to your SonarQube server.

  mvn sonar:sonar

Viewing Results:

- Navigate to your SonarQube dashboard (e.g., http://localhost:9000) and locate your project.

- You will find detailed code quality metrics, including the code coverage reported by JaCoCo.